Reducing AI Risk and Confirmation Fatigue

LabFlow AI · Response to pitch feedback · October 28, 2026

The feedback

AI extraction can be wrong. If the student has to confirm every measurement, confirming becomes a reflex: they tap “yes” without reading. When a report later contains a wrong number, the AI gets the blame.

> *Our answer is not more confirmations. It is to make each confirmation mean something, and to have code catch the errors a tired student will skip past.*

1. Confirm less, and make it count

If every card looks the same, people stop reading them.

- **Only interrupt when something looks off.** A plain reading such as “Ensayo 3, temperatura 65 grados”, where the number appears in the transcript and is in range, gets a light card. Anything ambiguous, out of range, or a correction gets a card that is clearly different and asks for more.
- **Put the value on the button.** “Guardar 73 %” instead of “Sí”. The student confirms a number, not a dialog. By voice, LabFlow reads it back: “setenta y tres por ciento, ¿lo guardo?”
- **Show what was said next to what was understood:** *“setenta y tres”* → **73 %**. Highlight only the parts the AI interpreted (units, number words, “coma” decimals).
- **Make flagged values take real effort.** When a warning is showing, a plain “yes” is not enough; the student repeats the value or picks from options. Effort goes where the risk is.

2. Let code do the checks, not the human

This is how we back up the promise of “no invented numbers”.

- **Grounding check (deterministic, no LLM):** every extracted number must trace back to the transcript, as digits or as Spanish number words. If the model outputs 73 and the transcript has neither “73” nor “setenta y tres”, it is blocked. This catches the worst failure: a number the model made up.
- **Range and unit warnings:** each variable has an expected range, and each value is compared with earlier trials (“this is 3× the previous yield”).
- **Unit normalization in code:** “grados” → °C and “por ciento” → % come from a lookup table. The model proposes; code checks.

3. Make every number traceable

Each measurement already keeps its original phrase, its source (voice or text), the time, and its state. We add:

- Who confirmed it, when, and whether they edited it or accepted it unchanged.
- The raw model output next to the final value.
- In the table and in any report, tapping a number shows: *"You said 'setenta y tres' at 14:02, LabFlow read 73 %, you confirmed it."* If a report is wrong, the record shows whether it was a transcription error or a confirmed misreading, instead of "the AI did it".

4. Reports: the AI writes the words, code does the math

- All statistics come from code querying the database. The LLM never calculates.
- After the AI writes, every number in the text is checked against the database. Any number without a match is rejected or flagged.
- The report stays labeled as a draft until the student signs off, and it lists any values that were flagged during capture.

5. A review step at the end of the session

- One focused moment instead of constant interruptions: "2 values were flagged, 1 was corrected. Review them before you leave the lab."

6. Measure it

- **Extraction evaluation set:** report a real number, e.g. "LabFlow extracts correctly in X of 50 dictations."
- **Watch for blind confirming:** confirmations faster than about one second, or never editing across many cards. Live data shows whether people actually read.

Priority for the Friday demo

Low cost and strong for the pitch:

1. The grounding check (section 2).
2. The value on the button, plus said vs. understood on the card (section 1).
3. The provenance view when tapping a number (section 3); most of the data is already stored.

One line for the pitch:

> *"LabFlow only interrupts when something looks wrong, every number traces back to what you said, and the AI never does the math."*